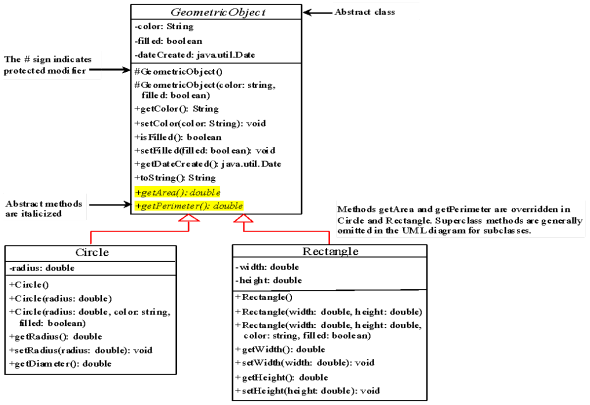
ชื่อ-นามสกุล.....ปิยดา บุตรศรีภูมิ.....รหัสประจำตัวนักศึกษา.....6804062612161..... *ปีการศึกษา 2569*

แบบฝึกหัดปฏิบัติการคาบที่ 8: Interface / Abstract

คำสั่ง

1. ให้ศึกษาหลักการ Interface / Abstract ต่อไปนี้
   เราสามารถที่จะนิยามเมธอดต่างๆ ภายในคลาสขึ้นมาก่อนได้โดยยังไม่ต้องกำหนดคำสั่งต่างๆภายในเมธอดเหล่านั้น แต่จะให้มีการกำหนดคำสั่งภายในคลาสอื่นๆ ที่จะสืบทอด วิธีการนี้จะช่วยทำให้เราสามารถจะกำหนดเมธอดที่ไม่สามารถนิยามคำสั่งไว้ล่วงหน้าได้ ภาษาจาวาได้กำหนดคลาสลักษณะนี้ไว้สองแบบคือ คลาสแบบ Abstract และ
   อินเตอร์เฟส



คลาสแบบ Abstract

- คลาสที่มีคำว่า abstract นำหน้าจะหมายความว่าคลาสนั้นยังเป็นคลาสที่ไม่สมบูรณ์ โดยมีเมธอดแบบ abstract ซึ่งเป็นเมธอดที่ยังไม่สมบูรณ์อย่างน้อยหนึ่งเมธอดอยู่ในคลาส
- abstract method จะต้องอยู่ในคลาสที่เป็น abstract class
- ถ้าคลาสลูกของ คลาสแม่ที่เป็น abstract ไม่เขียนรายละเอียดของ abstract methods ทั้งหมด ตัวของคลาสลูกเองต้องกำหนดเป็น abstract
- คลาสลูกที่ไม่ได้เป็น abstract ที่สืบทอดมาจากคลาสแม่ที่เป็น abstract **ทุกๆ abstract methods จะต้องถูกเขียนรายละเอียดการทำงาน**แม้ว่าจะไม่ได้ใช้ใน subclass ก็ตาม
- ไม่สามารถสร้างวัตถุจาก abstract class ด้วยโอเปอร์เรเตอร์ new แต่สามารถใช้ abstract class เพื่อเป็น data type เช่น GeometricObject[] geo = new GeometricObject[10];

- สามารถที่จะสร้าง constructors ได้โดยสามารถถูกเรียกได้จาก constructors ของคลาสลูก เช่น

  constructors ของ GeometricObject สามารถถูกเรียกจากคลาส Circle และคลาส Rectangle ได้
- สามารถสร้าง abstract class ที่ไม่มี abstract methods เลยก็ได้ในกรณีนี้เราไม่สามารถสร้างวัตถุจากคลาสดังกล่าวโดยการใช้โอเปอเรเตอร์ new โดยคลาสดังกล่าวจะถูกใช้เป็น base class เพื่อสร้าง subclass ตัวใหม่
- คลาสลูกสามารถเป็น abstract ได้แม้ว่า superclass จะเป็นคลาสปกติ (concrete class) เช่น

  abstract class GeometricObject extends Object
- คลาสลูกสามารถโอเวอร์ไรด์เมธอดจากคลาสแม่เพื่อให้เป็น abstract method ก็ได้

อินเตอร์เฟส อินเตอร์เฟส คือโครงสร้างที่มีลักษณะคล้ายคลาส ส่วนประกอบของอินเตอร์เฟสจะมีเฉพาะ ค่าคงที่ constants abstract methods

- จุดประสงค์ของการสร้าง interface จะคล้าย abstract class แต่จุดประสงค์หลักจริง ๆ คือการกำหนดพฤติกรรม behavior สำหรับ objects เช่น Comparable, Edible, Cloneable
- interface เป็นคลาสพิเศษในภาษาจาวาโดย ทุกๆ interface จะถูกคอมไพล์เป็น bytecode เหมือนคลาสปกติ
- ไม่สามารถสร้างวัตถุจาก interface โดยการใช้ new แต่สามารถใช้สำหรับการประกาศชนิดข้อมูลให้กับตัวแปร

2. ให้ศึกษาและทดลองพิมพ์ Code ตัวอย่างการสร้างอินเตอร์เฟสจากตัวอย่างต่อไปนี้

```
1 public class House implements Cloneable, Comparable<House> {
2    private int id;
3    private double area;
4    private java.util.Date whenBuilt;
5    public House(int id, double area) {
6       this.id = id;
7       this.area = area;
8       whenBuilt = new java.util.Date();
9    }
10   public int getId() {
11      return id;
12   }
13   public double getArea() {
14      return area;
15   }
16   public java.util.Date getWhenBuilt() {
17      return whenBuilt;
```

```
18    }
19    @Override
21    public Object clone() {
22       try {
23          return super.clone();
24       }
25       catch (CloneNotSupportedException ex) {
26          return null;
27        }
28    }
29    @Override // Implement the compareTo method defined in Comparable
30    public int compareTo(House o) {
31       if (area > o.area)
32          return 1;
33       else if (area < o.area)
34          return -1;
35       else
36          return 0;
37       }
38 }
```

2.1 ให้อธิบายการทำงานของโปรแกรมต่อไปนี้

บรรทัดที่ 1: **ประกาศคลาส** `House` **ที่ใช้งาน** (implements) **อินเตอร์เฟส** `Cloneable` **(เพื่อให้สามารถคัดลอกวัตถุได้) และ** `Comparable<House>` **(เพื่อให้สามารถเปรียบเทียบขนาดระหว่างวัตถุ** `House` **ได้)**

บรรทัดที่ 2- 4 : **ประกาศตัวแปรฟิลด์แบบ** `private` **ได้แก่** `id` **(รหัสบ้าน),** `area` **(พื้นที่บ้าน) และ** `whenBuilt` **(วันที่สร้างบ้าน)**

บรรทัดที่ 5- 9 : Constructor **สำหรับสร้างวัตถุ** `House` **โดยรับค่า** `id` **และ** `area` **เข้ามากำหนดให้ตัวแปรของวัตถุ พร้อมสร้างวัตถุ** `Date` **ปัจจุบันเก็บไว้ใน** `whenBuilt`

บรรทัดที่ 10-18 : **เมธอด** Getter (`getId()`, `getArea()`, `getWhenBuilt()`) **สำหรับดึงค่าตัวแปรฟิลด์ที่เป็น** `private`

บรรทัดที่ 21-28 : **เมธอด** `clone()` **ที่ทำการ** Override **จากคลาส** `Object` **เพื่อคัดลอกวัตถุ** `House` **โดยเรียกใช้** `super.clone()`

บรรทัดที่ 30-37 : **เมธอด** `compareTo(House o)` **ที่ทำการ** Implement **จาก** `Comparable` **เพื่อเปรียบเทียบพื้นที่บ้าน (**`area`**) โดยคืนค่า** `1` **ถ้าพื้นที่มากกว่า,** `-1` **ถ้าน้อยกว่า และ** `0` **ถ้าพื้นที่เท่ากัน**

2.2 สร้างคลาสทดสอบ และเพิ่มคำสั่งต่อไปนี้ใน main() ผลลัพธ์จะมีผลลัพธ์อย่างไร

```
House house1 = new House(1, 1750.50);
House house2 = (House)house1.clone();
```

วัตถุ `house2` **ถูกสร้างขึ้นมาใหม่ในหน่วยความจำโดยมีข้อมูล** `(id = 1, area = 1750.50, whenBuilt)` **เหมือนกับ** `house1` **ทุกประการ โดยที่** `house1 == house2` **จะคืนค่าเป็น** `false` **(เป็นคนละวัตถุกัน) แต่** `house1.compareTo(house2)` **จะคืนค่าเป็น** `0` **(พื้นที่เท่ากัน)**

3 จงเขียน class ที่วาดกราฟจากฟังก์ชันต่อไปนี้

```
abstract class AbstractDrawFunction extends JPanel
{
  /**Polygon to hold the points*/
  private Polygon p = new Polygon();
  /**Default constructor*/
  protected AbstractDrawFunction ()
  {
    drawFunction();
    setBackground(Color.white);
  }
  /**Draw the function*/
  public abstract double f(double x);
  /**Obtain points for x coordinates 100, 101, ..., 300*/
  public void drawFunction()
  {
    for (int x = -100; x <= 100; x++)
    {
      p.addPoint(x+200, 200-(int)f(x));
    }
  }
  /**Paint the function diagram*/
  public void paintComponent(Graphics g)
  {
    super.paintComponent(g);
    // Draw x axis
    g.drawLine(10, 200, 390, 200);
   // Draw y axis
    g.drawLine(200,30, 200, 390);
    // Draw arrows on x axis
    g.drawLine(390, 200, 370, 190);
    g.drawLine(390, 200, 370, 210);
   // Draw arrows on y axis
    g.drawLine(200, 30, 190, 50);
    g.drawLine(200, 30, 210, 50);
    // Draw x, y
    g.drawString("X", 370, 170);
    g.drawString("Y", 220, 40);
    // Draw a polygon line by connecting the points in the polygon
    g.drawPolyline(p.xpoints, p.ypoints, p.npoints);
  }
}
```